



Back Propagation Neural Network

CCS3600 Artificial Intelligence — Enhanced Teaching Materials



What You Will Learn

Learning objectives for this module



Fundamentals

Understand the architecture and mechanics of feed-forward neural networks and how information flows through layers



Algorithm

Master the back-propagation algorithm, its mathematical foundation including loss functions, chain rule, and gradient computation



Applications

Explore real-world uses: pattern recognition, face recognition with CNNs, and speech recognition systems



Evaluation

Understand the history, key points, advantages, and limitations of backpropagation in modern AI

What is Backpropagation?



Short for
"Backward Propagation
of Errors"

Forward Pass



Backward Pass (Error)

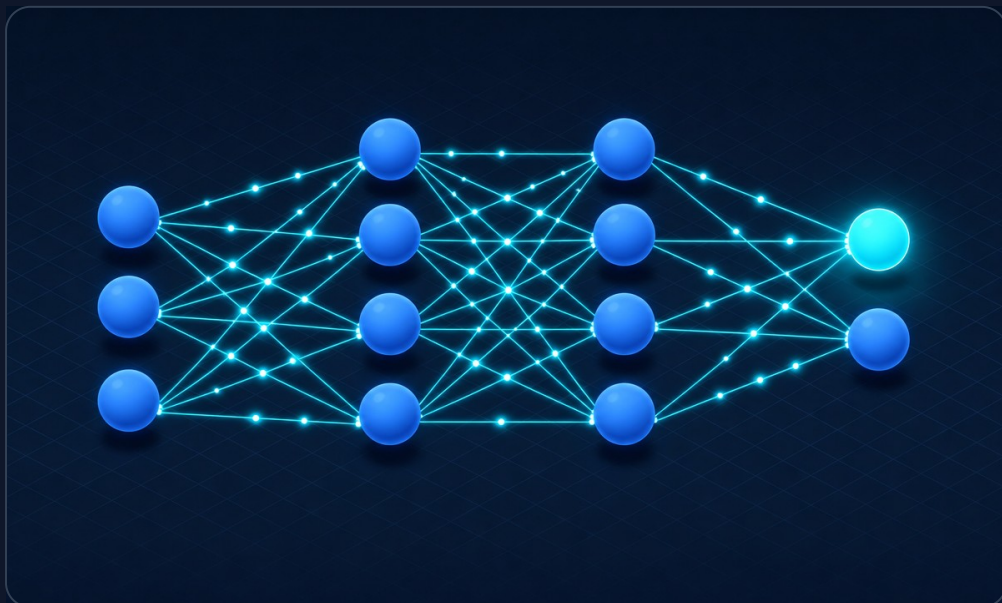


- ✓ **Fine-tuning Neural Network Weights**
Adjusts weights based on error rates from previous iterations (epochs) to reduce errors
- ✓ **Gradient Calculation**
Calculates the gradient of the loss function with respect to each weight in the network
- ✓ **Supervised Learning Foundation**
Key component for training feedforward neural networks using stochastic gradient descent
- ✓ **Individual Weight Updates**
Each weight updated individually to gradually reduce the loss function over training iterations
- ✓ **Deep Learning Enabler**
Enabled the surge in popularity of deep learning algorithms since the early 2000s

Backpropagation proceeds backwards through the network, from the last layer to the first.

Feed Forward Neural Networks

The foundation architecture for backpropagation



Key Characteristics

➡ No Cycles

Connections between nodes do not form a cycle

➡ One Direction Only

Data always moves forward, never backwards

➡ Simplest Form

Foundation model: single layer perceptron for classification

Feed Forward

Directed acyclic graph

One-way data flow

Static input → output

vs

Recurrent

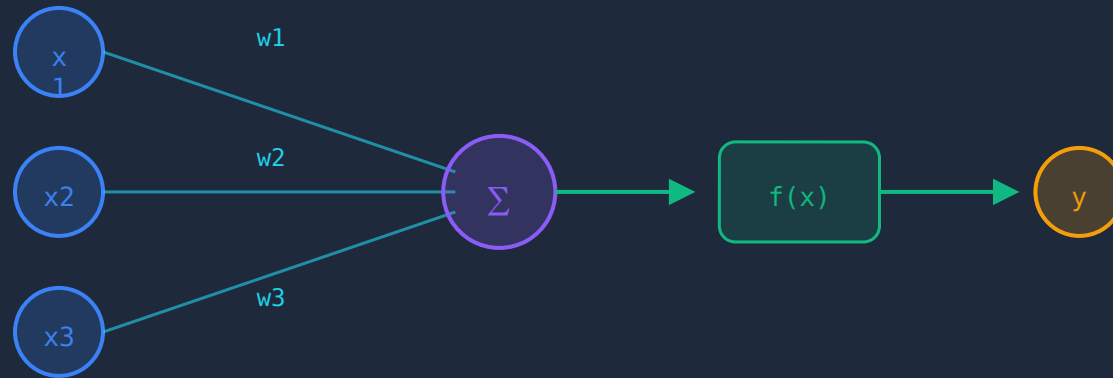
Contains cycles

Feedback loops

Sequential / time-series

How Feed Forward Networks Work

Single Layer Perceptron Model



Process:

1. Inputs multiplied by weights
2. Weighted values summed
3. Activation function applied
4. If $\text{sum} > \text{threshold}$: output = 1
If $\text{sum} < \text{threshold}$: output = -1

Delta Rule

Compare outputs with intended values to adjust weights through training. This produces a form of gradient descent.

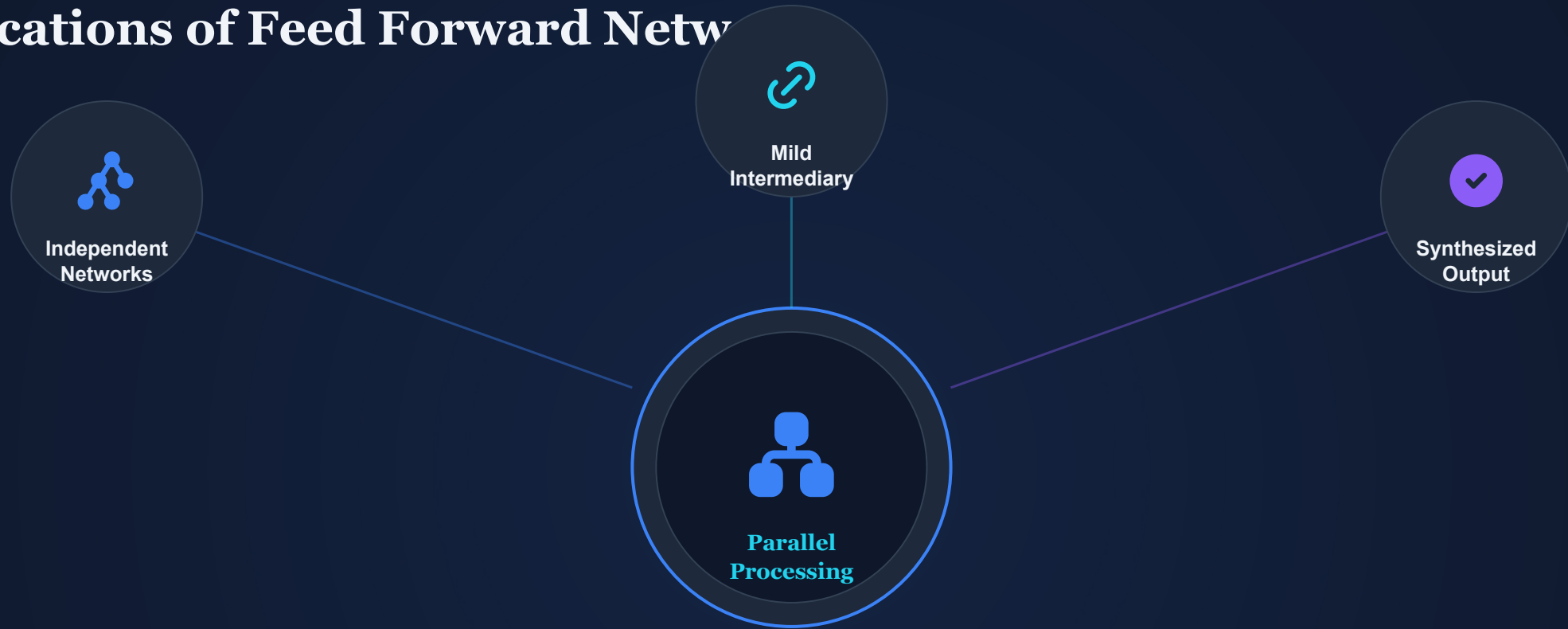
In multi-layered perceptrons, this process is defined as backpropagation.

Multi-Layer Perceptrons

Each hidden layer adjusted according to output values from the final layer.

Weight updating follows the same principle but propagates error backward through all intermediate layers.

Applications of Feed Forward Netw



Like the Human Brain

Multiple independent neurons handle and process larger tasks. Individual networks perform their tasks independently, and results are combined at the end to produce a synthesized, cohesive output — mimicking biological neural parallel processing.

Types of Backpropagation Networks

1

Static Backpropagation

Maps static input to static output.

Used for static classification tasks like optical character recognition (OCR).



Classification Tasks



2

Recurrent Backpropagation

Processes input until a fixed value is reached.

Used for sequence-dependent tasks where order matters and output feeds back.



Sequence-Dependent Tasks



3

BP Through Time (BPTT)

Handles time-series data by unrolling recurrent networks into feedforward form for training. Each timestep = a network copy.



Time-Series Data



Backpropagation Formula

Layer Output Equations

$$\text{Layer 1: } h_1 = f_1(W_1 \cdot x + b_1)$$

$$\text{Layer 2: } h_2 = f_2(W_2 \cdot h_1 + b_2)$$

$$\text{Layer N: } h_N = f_N(W_N \cdot h_{N-1} + b_N)$$

Symbol Reference

h_i	Output of hidden layer i
f_i	Activation function (sigmoid, ReLU, tanh)
W_i	Hidden weights matrix in layer i
x	Input vector to the neural network
N	Number of layers in the network
b_i	Bias matrix in layer i



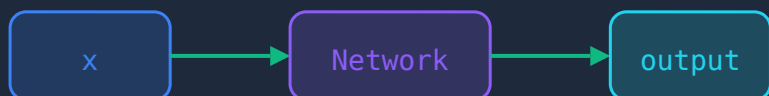
How It All Connects

Each layer receives the previous layer's output as input. The entire network is a composition of functions: the output of each layer feeds into the next. This chained structure is what makes backpropagation possible — the chain rule can be applied to compute gradients efficiently, layer by layer, from the output back to the input.

Loss Function for Backpropagation

➡ Forward Propagation

Input x passes through layers to produce output. Information flows forward.



🏠 Loss Function C

Measures how far predictions are from true labels:

$$C = \text{Loss}(\text{output}, \text{label})$$

Low C = good prediction
High C = poor prediction

Training Behavior



Common Loss Functions

Cross-Entropy

Cosine
Similarity

Hinge
Loss

Mean Squared Error

Objective: reduce C by adjusting network weights

Computing the Loss Gradient



Weight Notation

$w_{i,j,k}$ = weight from node j in layer (i-1) to node k in layer i. Each weight is updated individually to reduce C.

Chain Rule for Gradient Calculation

The Chain Rule

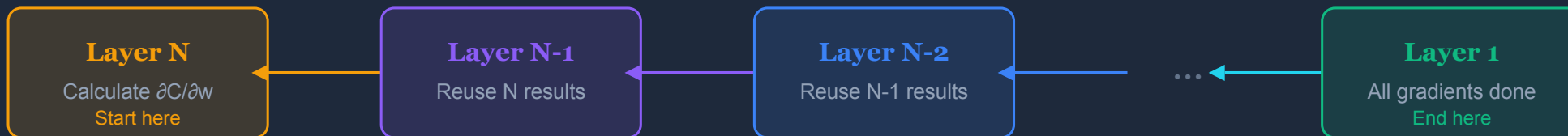
C depends on weight values via a chain of many functions. For z depending on y, where y depends on x:

$$dz/dx = (dz/dy) \cdot (dy/dx)$$

Key Efficiency Advantage

Derivatives at layer N calculated first. These are ingredients for layer N-1. Save and reuse derivative calculations via the chain rule — avoids duplicate calculations across all layers.

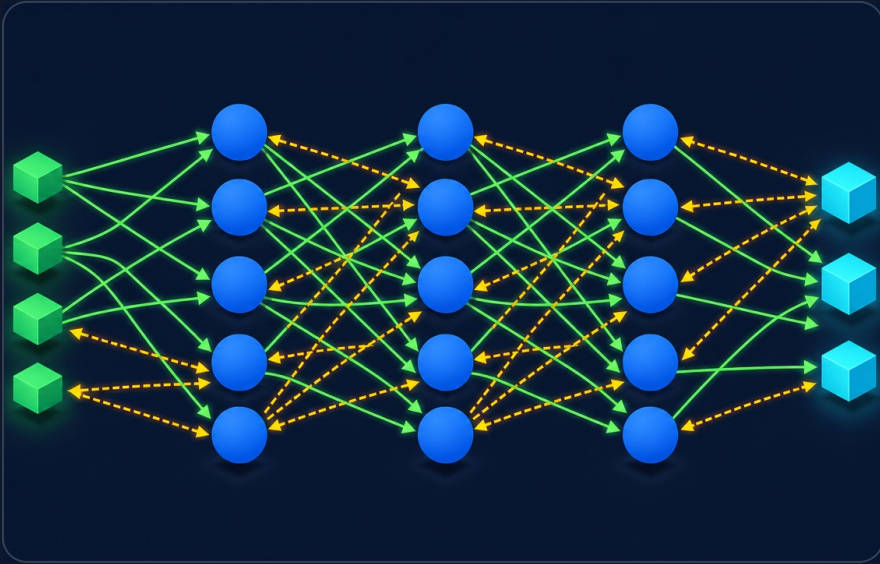
Backward Computation Flow



← Direction of gradient computation (backward through the network)

Each layer's derivatives are an ingredient in the chain rule formula for the previous layer.

How Backpropagation Algorithm Works



Algorithm Steps

- 1 Inputs X arrive through pre-connected path
- 2 Input modeled using randomly selected weights W
- 3 Calculate output for every neuron layer by layer
- 4 Calculate error in the outputs
- 5 Travel back, adjust weights to decrease error
- 6 Repeat until desired output is achieved

Error Calculation

$$\text{Error} = \frac{\text{Actual Output} - \text{Desired Output}}{\text{Output}}$$

≡ Computes gradient via chain rule

One layer at a time, efficiently

≡ Does not define gradient usage

Separate from optimization method

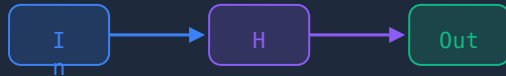
≡ Generalizes the delta rule

Works with any number of outputs

Standard BP vs Backpropagation Through Time

Standard Backpropagation

Applies to feedforward networks (directed acyclic graph)
Chain rule applied recursively through layers
Static input-output mapping, no temporal dependency



Backpropagation Through Time

"Unrolls" recurrent neural network across time steps
Treats unrolled RNN as feedforward for training
Handles sequential, time-dependent data



BPTT Challenges

Large unrolled networks become difficult to train
Easily trapped in local optima during optimization

Vanishing Gradient Problem

Distant interactions produce diminishingly small gradients
Makes learning long-range dependencies very difficult

Solutions

ReLU Activation

Regularization Techniques

LSTM / GRU Architectures



Why We Need Backpropagation

The essential algorithm that makes deep learning possible

Simplifies Structure

Removes least-effective weighted links, streamlining the network for better performance

Reveals Relationships

Uncovers the relationship between input layers and hidden unit layers during training

Assesses Impact

Measures how each input variable influences the final network output accuracy

Deep Network Essential

Critical for deep neural networks in error-prone and complex real-world projects

Mathematical Elegance

Leverages chain and power rules for efficient gradient computation across any number of outputs

Computational Necessity

Without backpropagation, training deep networks would be computationally infeasible

PART III

Real-World Applications

Pattern Recognition, Face Recognition, and Speech Recognition



👁 Pattern Recognition

What is a Pattern?

A set of objects or phenomena where the elements share similar traits or characteristics.

Biological Examples:

Paramecium, Amoeba, Euglena

Celestial Examples:

Star constellations in night sky

Human Perception

Humans naturally find patterns even in completely random noise and visual data.

Pareidolia Effect

Perceiving meaningful shapes in clouds, rock formations, noise

⚠ Can lead to false positives

Subjective, inconsistent results

Neural Network Approach

Neural networks trained via BP excel at automated, consistent pattern recognition.

✓ Objective and repeatable

✓ Scales to millions of samples

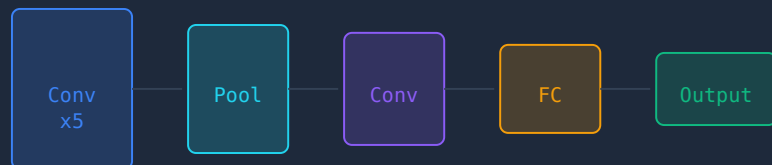
✓ Learns features automatically

✓ Real-time classification

Backpropagation in Face Recognition

CNN Architecture (VGGFace)

Parkhi, Vidal & Zisserman (2015)



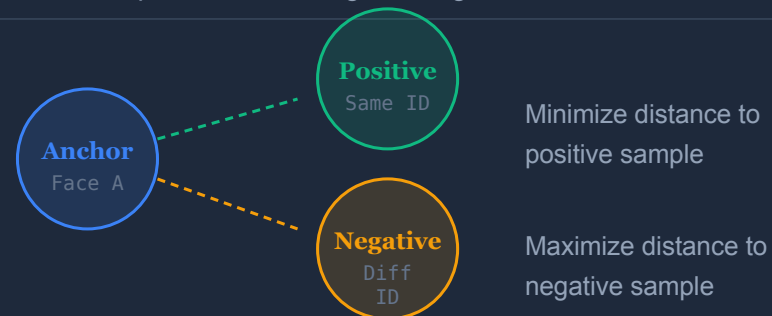
18 layers total | 2,622 identity classes | 2.6M face images

18-layer Deep CNN

Celebrity face training data

Triplet Loss Function

Refinement technique for embedding learning



$$L = \max(d(a,p) - d(a,n) + \text{margin}, 0)$$

How Backpropagation Drives Face Recognition Training

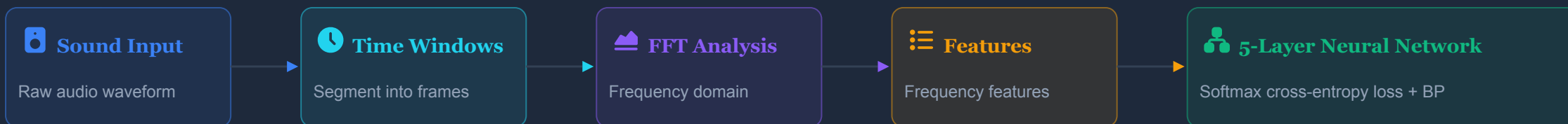


🎤 Backpropagation for Speech Recognition

🏠 Sony Corporation Research

English and Japanese speech recognition on embedded devices

Speech Processing Pipeline



Network Architecture

5-layer deep neural network
optimized for embedded devices

$$\text{Loss} = -\sum(y * \log(y_{\text{hat}}))$$

Softmax cross-entropy for multi-class output

Transfer Learning

Train on Japanese data first,
then transfer learned features
to English recognition model.

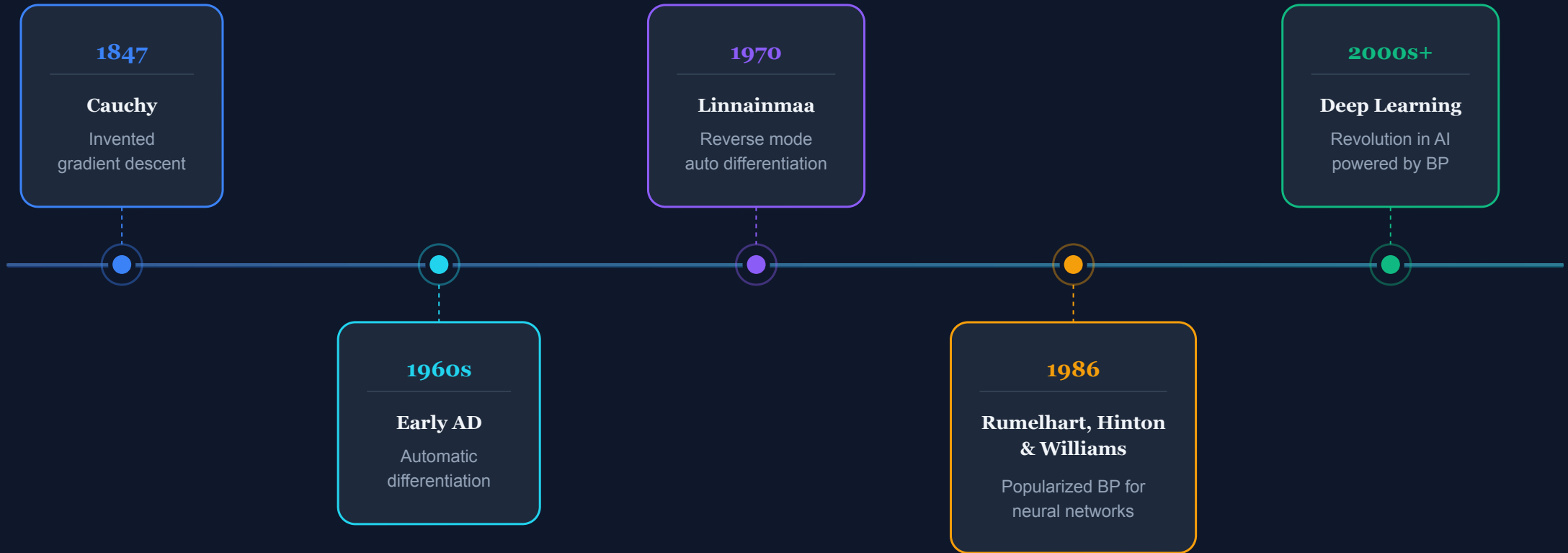


BP Role in Training

Backpropagation adjusts weights
across all 5 layers by computing
gradients of the cross-entropy
loss with respect to each weight.

✔ Enables embedded device deployment

🕒 History of Backpropagation



Over 170 Years of Mathematical Foundations

From Cauchy's gradient descent to modern deep learning, backpropagation stands on centuries of mathematical progress

★ Key Points & Best Practices

1 Simplify Network Structure

Remove least-effective weighted links to create a leaner, more efficient network architecture.

2 Study Layer Relationships

Examine input and activation values to understand hidden relationships between layers.

3 Assess Input Variable Impact

Measure how each input variable influences the final network output and prediction accuracy.

4 Deep NN Applications

Especially useful for deep neural networks in image recognition, speech processing, and NLP tasks.

5 The "Shoe Lace" Analogy

Like tying a shoe lace, backpropagation is an iterative process of adjustment.

Each iteration fine-tunes the weights until the network produces the correct output.

The process repeats: forward pass, compute error, backward pass, update weights.



★ Summary

Backpropagation is the cornerstone of deep learning training. It enables efficient gradient computation across arbitrarily deep networks, making modern AI applications in vision, speech, and language understanding possible.

⚠ Disadvantages of Backpropagation

⚠ Data Quality Dependency

Performance is heavily dependent on the quality of input data. Poor or biased training data leads to inaccurate weight adjustments and unreliable models.

⚠ Noise Sensitivity

The algorithm is highly sensitive to noisy data. Noise in training samples can propagate through the network and degrade overall learning effectiveness.

⚠ Batch Strategy Trade-offs

Full matrix-based approach is preferred over mini-batch for accuracy, but requires significantly more memory and computation per update step.

⚠ Vanishing / Exploding Gradients

Gradients can become vanishingly small or explosively large in deep networks, making training unstable or impossible for certain architectures.

⚠ Computational Expense

For large networks with millions of parameters, backpropagation becomes computationally expensive. Each training iteration requires a full forward and backward pass, scaling with network depth and width.

Thank You

Backpropagation in Neural Networks

References

- DeepAI.org
deepai.org/machine-learning-glossary-and-terms/backpropagation
 - Guru99.com
guru99.com/backpropagation-neural-network.html
-
- Original Lecture: Maslina @ SSK5603
 - Enhanced with AI tools for CCS3600 Mini Project

CCS3600 Artificial Intelligence

